Feature request: Bulk batch operations (webhook resend, cleanup, empty run restart)

To: ValueSERP / Traject Data Product & API teams
From: Semify

Date: March 30, 2026
Subject: Proposal for account level batch maintenance APIs

1. Executive summary

We integrate deeply with the ValueSERP Batches API using notification webhooks to ingest result sets when batches complete. At scale, routine maintenance recovering from failed webhook deliveries, clearing batches that have already been processed successfully, and rerunning batches that produced no usable results currently requires many sequential API calls (list batches → list result sets → resend webhook, delete, or start per batch).

We are requesting first-class, filterable bulk (or batch job) operations so customers can perform these tasks safely, observably, and within documented rate limits, without building fragile client-side loops.

This document summarizes business needs, gaps in today's public API, and concrete API proposals for your team to evaluate.

2. Background & how we use ValueSERP

- We create manual batches, attach searches, call start, and rely on HTTPS webhooks when result sets complete (webhook documentation).
- We download JSON result sets and process them in our application , we may delete batches at ValueSERP after successful processing (delete batch).
- Operational issues (our downtime, network blips, webhook timeouts) cause failed webhook deliveries , ValueSERP already exposes per–result set webhook resend and result metadata including webhook status (list result sets).

Pain point: For hundreds or thousands of batches, per entity calls are slow, error prone, and expensive to orchestrate under list rate limits (e.g. 60 requests/minute for batch list).

3. Current API capabilities (relevant endpoints)

Capability	Endpoint(s)	Limitation for our use case
List batches (filters, pagination)	GET /batches	No filter for “webhook failed” or “safe to delete after success.”
List result sets + webhook status	GET /batches/{id}/results	We must call per batch to inspect webhook outcome.
Resend webhook	GET /batches/{id}/results/{result_set_id}/resendwebhook	Single result set only.
Delete batch	DELETE /batches/{id} (force optional)	Single batch only.
Start / re run batch	GET /batches/{id}/start	Single batch only.

Conclusion: The building blocks exist, account level or filtered bulk operations do not appear in the public documentation.

4. Business outcomes we are trying to achieve

1. Reliability: Quickly redeliver webhooks for all failed deliveries in a time window (disaster recovery, post outage catches up).
2. Hygiene: Remove batches from the ValueSERP dashboard after we have confirmed successful webhook delivery and local processing without manually deleting thousands of batches one by one.
3. Quality: Rerun batches that were completed but returned no effective search results (zero or empty result payload where we expected data), without scripting fragile heuristics client-side.

5. Proposed features (high level)

5.1 Bulk (or job based) “resend failed webhooks”

Goal: One request to enqueue (or execute) webhook resend for all result sets matching criteria such as:

- Webhook delivery failed (using the same notion of failure as in webhook status on result sets), and/or
- Optional time bounds (created after / created before on batch or result set), and/or
- Explicit allowlist of batch_id / result set pairs (with caps).

Why server side: Centralized validation, fair queuing, and clear audit / job status for large backlogs.

5.2 Bulk (or job based) “delete batches we are done with”

Goal: Delete batches that are safe to remove, for example where the latest result set shows successful webhook delivery to our endpoint optionally constrained by creation date and excluding running batches.

Requirements to clarify in design:

- Exact rule for “webhook succeeded” (align with existing webhook status on list result sets).
- Interaction with existing force=true behavior when webhook had failed (delete batch).
- dry run mode returning count and/or batch IDs without deleting.

5.3 Bulk (or job based) “restart batches with empty / unusable runs”

Goal: Identify batches that ran but produced no completed searches or no usable result data (exact definition is product dependent), and start them again (start batch), respecting credits and not starting batches already running.

Why: Reduces manual triage when result sets exist but searches completed is zero or results are empty due to transient platform issues.

6. Suggested API shape (for discussion not prescriptive)

The examples below are illustrative only, naming, nesting, and HTTP verbs should follow Traject ValueSERP's existing Batches API conventions (not our internal preferences).

How today's public API is structured (for alignment)

- Host and base path: https://api.valueserp.com/batches (see List Batches).
- api_key is passed as a querystring parameter on the URL in documented examples.
- Path segments are lowercase, nested resources use plural nouns where applicable (results, searches), e.g. GET /batches/{BATCH_ID}/results, GET /batches/{BATCH_ID}/searches/{PAGE}.
- Compound path segments appear without hyphens (e.g. resendwebhook on GET /batches/{BATCH_ID}/results/{RESULT_SET_ID}/resendwebhook per Resend Webhook) , single batch delete uses DELETE /batches/{BATCH_ID} (Delete Batch) , single batch run uses GET /batches/{BATCH_ID}/start (Start Batch).

Illustrative bulk operations (Traject style paths final design is yours)

We assume account scoped operations remain under the /batches tree and use noun-like segment names consistent with the patterns above (lowercase, no hyphenated segments unless you introduce them consistently across the API).

Use case	Illustrative creation (submit work)	Illustrative read (job / request status)	Notes
Redeliver webhooks for many result sets that match criteria (equivalent to many calls to …/resendwebhook)	POST /batches/webhookresends	GET /batches/jobs/{JOB_ID}	Name parallels the existing resendwebhook segment , body/query: selection filters, caps. Response returns a job_id (or equivalent) for polling.

Remove many batches that match “safe to delete” rules (equivalent to many DELETE /batches/{BATCH_ID})	POST /batches/deletions	GET /batches/jobs/{JOB_ID}	Aligns with delete semantics and optional force from the existing delete endpoint.
Start many batches whose last run was empty / unusable (equivalent to many GET /batches/{BATCH_ID}/start)	POST /batches/starts	GET /batches/jobs/{JOB_ID}	Mirrors the existing start segment, search credits should match current start behavior.

Job resource

- A shared /batches/jobs/{JOB_ID} (or a name you prefer: e.g. batchjobs) would match the pattern of nesting collections under /batches the same way results and searches are nested today.
- Alternatively, status could live under each operation (e.g. GET /batches/webhookresends/{JOB_ID}) if that fits your routing better we defer to your routing and naming standards.

HTTP methods

- Today, resendwebhook and start are documented as GET, delete uses DELETE. For large selection payloads, POST with a JSON body is often appropriate please choose what matches Traject’s API guidelines and security posture.
- For long running work, 202 Accepted plus polling GET /batches/jobs/{JOB_ID} (or equivalent) is a common pattern.

Rate limits & quotas

- Document interaction with existing limits (e.g. batch list rate limits) and search credit consumption for operations equivalent to start.

7. Success criteria (from our side)

- We can operate on many batches in one controlled operation with clear success/failure reporting.
- Behavior is documented (OpenAPI or equivalent), idempotent where possible, and safe by default (dry run, caps, explicit filters).
- No ambiguity vs. existing endpoints: same webhook payload, same delete semantics, same start/charging rules.

8. Nongoals (optional)

- Changing core webhook payload format (unless you propose improvements separately).
- Replacing per batch APIs those remain essential for narrow debugging.

9. References (ValueSERP public documentation)

- Batches API Overview
- List Batches
- Webhook
- List Result Sets
- Resend Webhook
- Delete Batch
- Start Batch